

AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING

International Credit Hour Engineering Programs (ICHEP)
Computer Engineering and Software Systems Program (CESS)



CSE351 / CSE361

Computer Networks



Mini RFC
Team 25



AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
International Credit Hour Programs (ICHEP)
Computer Engineering and Software Systems Program (CESS)

Computer Networks

Multi-Player Game State Synchronization

Team 25

Done by:	Ahmed Hazem Khalil	22P0226
	Karim Hatem	22P0153
	Ahmed Aabdallah Nofull	22P0255
	Mohamed Amir Kamal	2301144
	Youssef Maged Mores	2301081
	Marwan Wagih Mohamed	23P0247

Delivered to:

Dr. Karim Emara
Dr. Ayman Bahaa Eldin
Eng. Mina

Table of Contents

1.0 Introduction	3
2.0 Protocol Header Format	3
2.1 Entities	3
2.2 Data Flow	4
2.3 Sequence Flow.....	5
2.4 Client Finite State Machine	6
3.0 Message Types	7
4.0 Communication Procedures.....	7
4.1 Session Start (Connection Establishment).....	7
4.2 Normal Data Exchange.....	7
4.3 Error Recovery	7
4.4 Shutdown	7
5.0 Reliability & Performance Features	8
5.1 Selective Reliability Model	8
5.2 Retransmission Strategy	8
5.3 Delta Compression & Bandwidth Optimization.....	8
5.4 Sequence Manager & Sliding Window	8
6.0 Experimental Evaluation Plan	8
6.1 Baseline Execution	8
6.2 Network Impairment Simulation (Linux NetEm).....	9
6.3 Metrics Collected.....	9
6.4 Automation	9
7.0 Example Use Case Walkthrough.....	10

Table of Figures

Figure (1): Data Flow	4
Figure (2): Sequence Flow	5
Figure (3): Client Finite State Machine.....	6
Figure (4): Netem Output	9
Figure (5): Server Log Output.....	10
Figure (6): Wireshark Screenshot.....	10

MINI RFC 2025-GUDP: Real-Time Grid Clash Protocol Specification

Status: Experimental

Version: 2.0

Date: December 2025

Author: Team 25

1.0 Introduction

The **GridClashSync** phase one protocol (also referred to as **Gaming User Datagram Protocol (GUDP)** phase two) is a custom application-layer protocol designed for real-time multiplayer game state synchronization in the *Grid Clash* environment. It operates over UDP to minimize latency while implementing selective reliability for critical game events. The protocol follows an authoritative server architecture with client-side prediction, delta compression, and jitter compensation.

Core Design Constraints

- UDP transport (no TCP retransmissions)
- Total packet size ≤ 1200 bytes
- Target end-to-end latency ≤ 50 ms
- Supports up to 4 concurrent clients
- Operates correctly under 2–5% packet loss

Evolution from Phase 1 to Phase 2

Phase 1 focused on a basic UDP-based sync mechanism with minimal reliability. Phase 2 introduced **selective reliability, delta compression, and enhanced header fields** to improve performance under packet loss and reduce bandwidth usage.

2.0 Protocol Header Format

Every GUDP packet begins with a fixed **28-byte** header utilizing Big-Endian byte order to ensure cross-platform compatibility. Checksum was added in phase 2.

2.1 Entities

- **Server:** the central authority responsible for keeping the authoritative grid state. It collects player actions, updates the global game map, and broadcasts periodic snapshots.
- **Clients:** individual player applications that send user actions (clicks or moves) to the server and apply the updates received in snapshots.

2.2 Data Flow

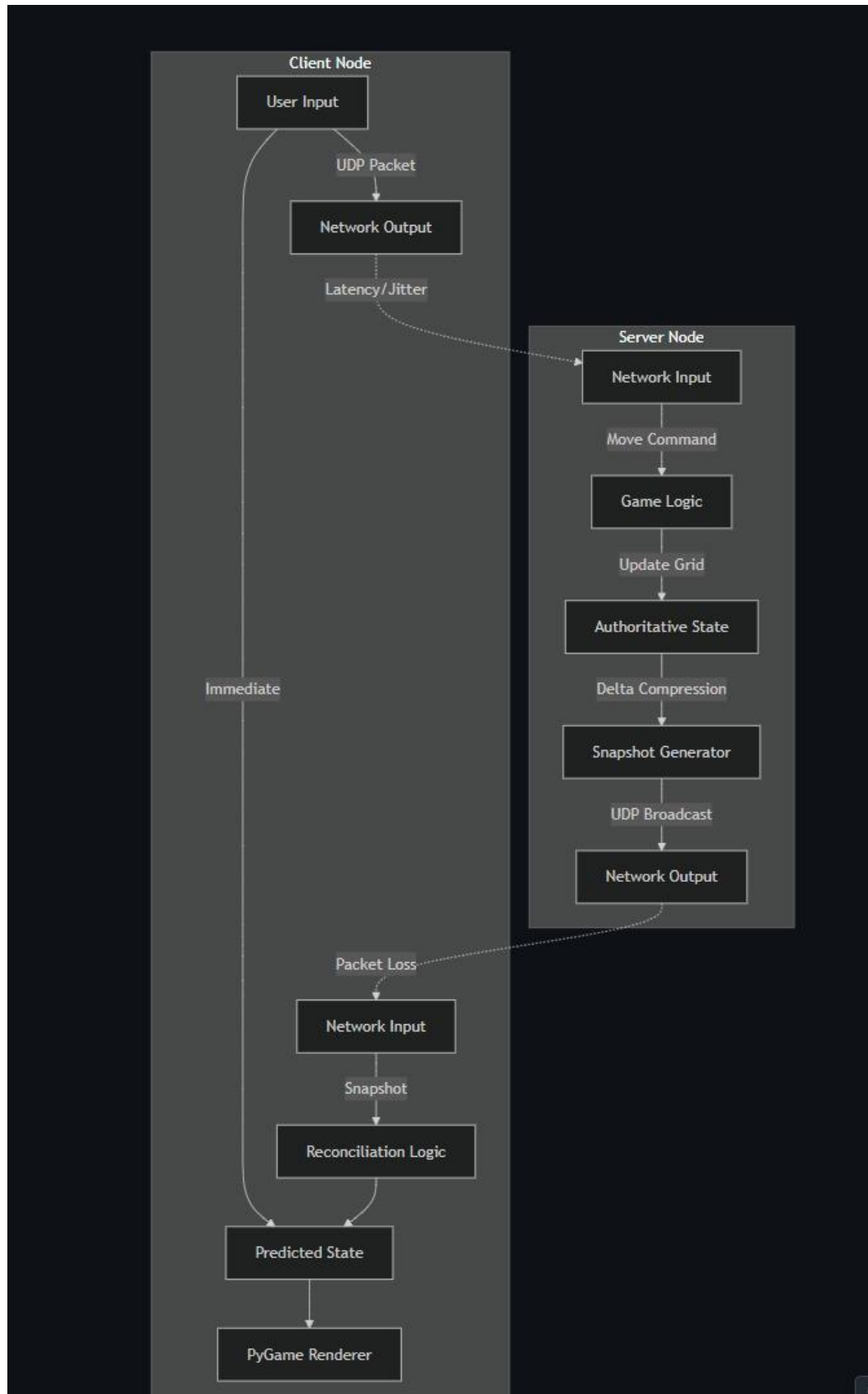


Figure (1): Data Flow

2.3 Sequence Flow

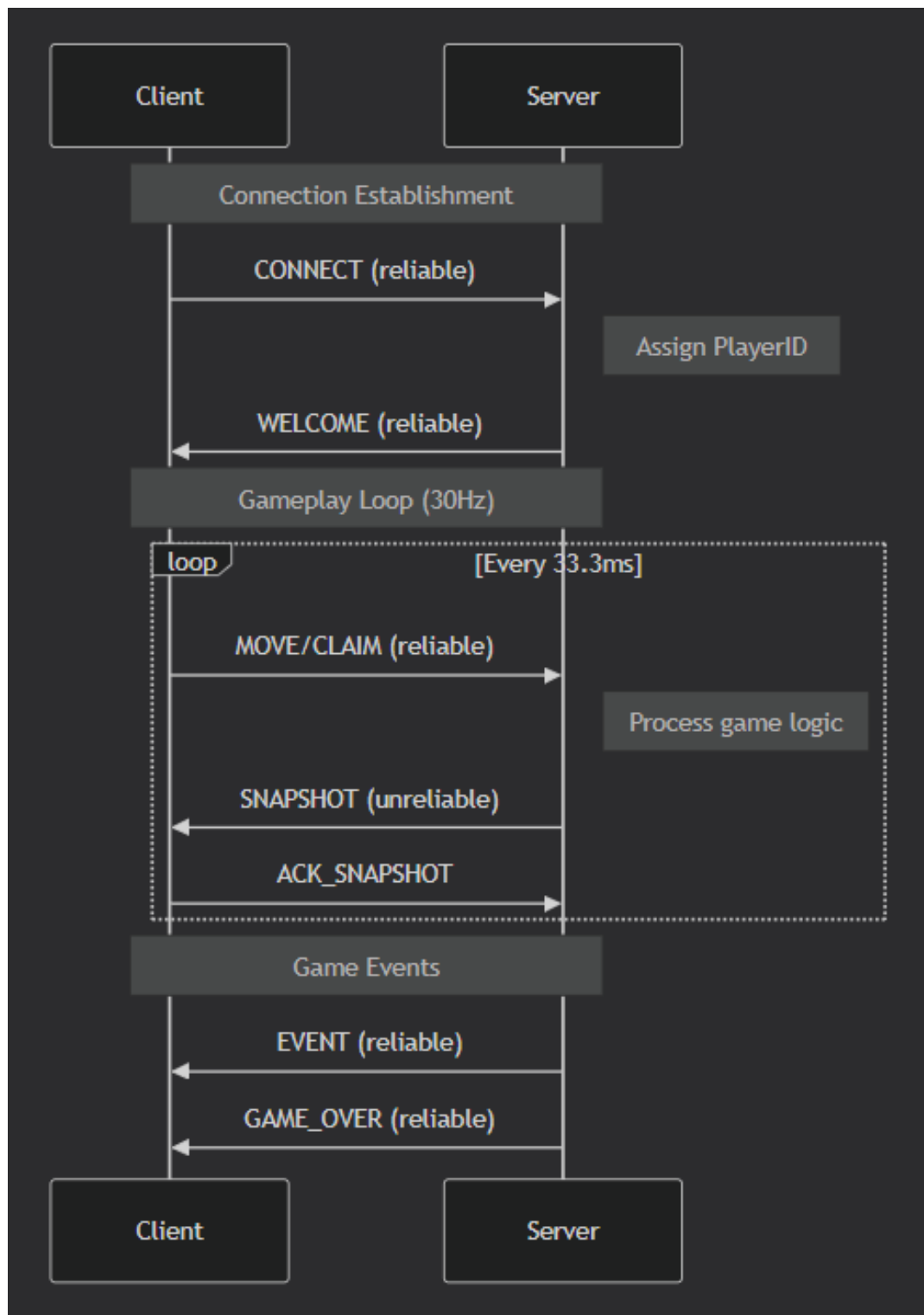


Figure (2): Sequence Flow

2.4 Client Finite State Machine

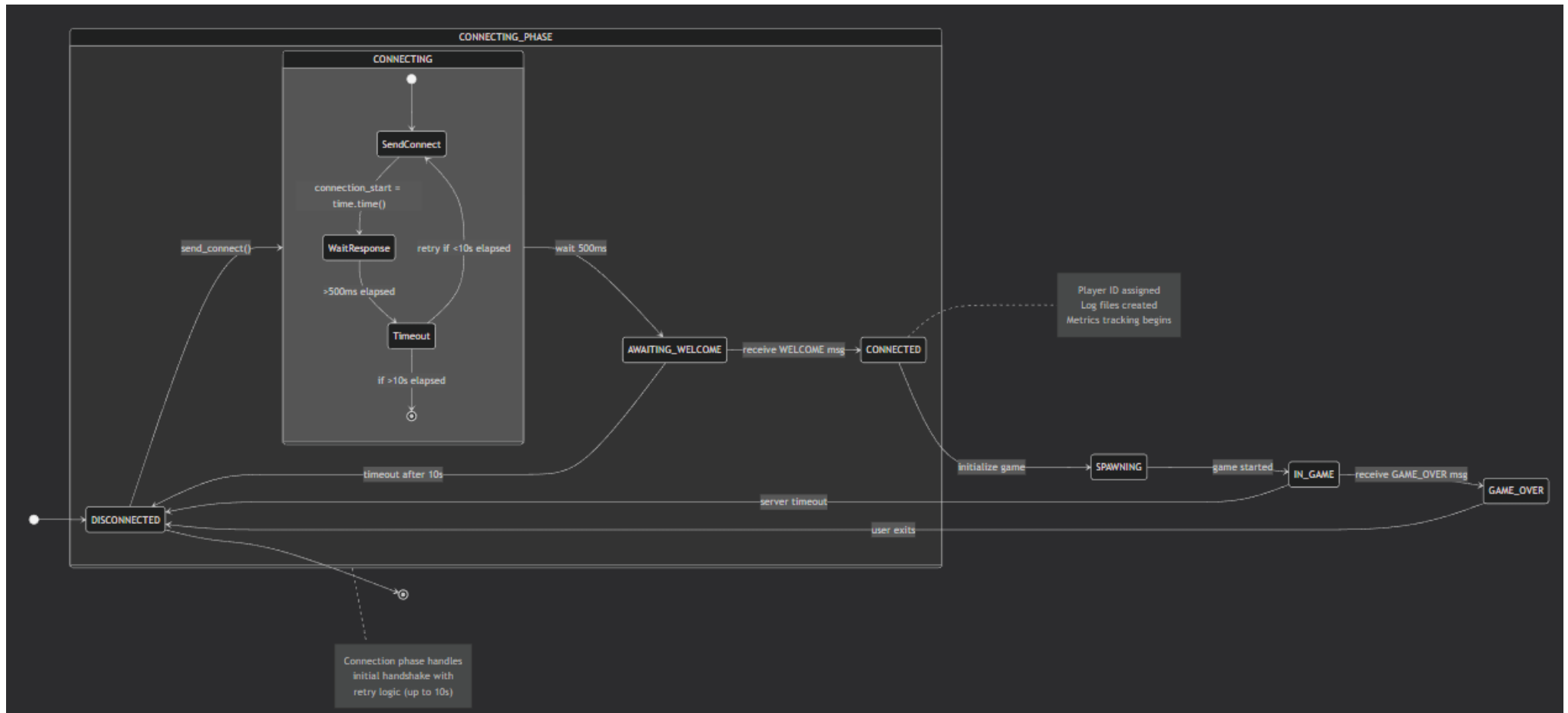


Figure (3): Client Finite State Machine

3.0 Message Types

Phase 1 defined 4 basic message types. Phase 2 expanded this to support richer gameplay and reliability mechanisms.

Offset (Bytes)	Field Name	Data Type	Python Fmt	Description	Justification
0-3	PROTO_ID	char[4]	>4s	Literal b'GUDP'	Magic bytes to reject non-protocol traffic on the port.
4	VERSION	uint8	B	2	Allows backward compatibility if packet structure changes.
5	MSG_TYPE	uint8	B	Enum (0-14)	Identifies payload handler (e.g., Move vs. Snapshot).
6-9	SNAPSHOT_ID	uint32	I	Integer	Global game state counter. Used for delta encoding reference.
10-13	SEQ_NUM	uint32	I	Integer	Per-client sequence number for packet loss detection and ordering.
14-21	TIMESTAMP	uint64	Q	Integer	Unix timestamp (ms). Used for RTT/Latency calculation.
22-23	PAYLOAD_LEN	uint16	H	Integer	Size of the data following the header (Max 65535).
24-27	CHECKSUM	uint32	I	Integer	CRC32 of the payload. Ensure data integrity against corruption.

Phase 2 introduced gameplay events, acknowledgments, and compression support to improve gameplay experience and network efficiency.

4.0 Communication Procedures

4.1 Session Start (Connection Establishment)

1. Client sends **CONNECT** (type 0, reliable).
2. Server replies with **WELCOME** (type 1, reliable) containing playerID and grid dimensions.
3. Client enters **CONNECTED** state.

4.2 Normal Data Exchange

- **Client → Server:** MOVE, CLAIM (reliable).
- **Server → Clients:** SNAPSHOT (unreliable, delta-compressed).
- **Bidirectional:** HEARTBEAT (unreliable), ACK/NACK (reliable).

4.3 Error Recovery

- Missing reliable packets trigger **RESEND_REQUEST**.
- Server may resend **WELCOME** or **GAME_OVER** if no ACK received.
- Timeout = 2 * **UPDATE_INTERVAL** (≈ 66 ms).

4.4 Shutdown

Server sends **GAME_OVER** (reliable). Clients may send final ACK and close socket.

5.0 Reliability & Performance Features

5.1 Selective Reliability Model

- **Reliable:** CONNECT, WELCOME, MOVE, CLAIM, GAME_OVER, ACK, NACK.
- **Unreliable:** SNAPSHOT, HEARTBEAT, COMPRESSED.

5.2 Retransmission Strategy

- Reliable packets use **stop-and-wait** with max 3 retransmissions.
- Retransmission timeout (RTO) = $2 * \text{average_RTT}$ (empirically set to 66 ms).

5.3 Delta Compression & Bandwidth Optimization

- Snapshots sent at 30 Hz.
- If state hash changes, only changed cells are sent.
- Payload > 200 bytes → Zlib compression (MSG_TYPE = COMPRESSED).

5.4 Sequence Manager & Sliding Window

- Window size = 32 packets.
- Bitmask tracks received packets for fast loss detection.

6.0 Experimental Evaluation Plan

6.1 Baseline Execution

- Script: baseline.py
- Launches 1 server + 4 headless clients.
- Runs for 40 seconds each for every error and under normal conditions.

6.2 Network Impairment Simulation (Linux NetEm)

```
=====
Running scenario: baseline
=====
[sudo] password for karim:
No netem (baseline).
Starting tcpdump capture to pcaps/baseline.pcapng ...
Game running for 40 seconds...
Stopping clients and server...
Run complete. CSV logs should be created.
Stopping tcpdump ...
Running analysis with process_logs.py ...
Scenario baseline finished. Summary in baseline_analysis.txt

=====
Running scenario: loss2
=====
Applying netem: loss 2%
Starting tcpdump capture to pcaps/loss2.pcapng ...
Game running for 40 seconds...
Stopping clients and server...
Run complete. CSV logs should be created.
Stopping tcpdump ...
Running analysis with process_logs.py ...
Scenario loss2 finished. Summary in loss2_analysis.txt

=====
Running scenario: loss5
=====
Applying netem: loss 5%
Starting tcpdump capture to pcaps/loss5.pcapng ...
Game running for 40 seconds...
Stopping clients and server...
Run complete. CSV logs should be created.
Stopping tcpdump ...
Running analysis with process_logs.py ...
Scenario loss5 finished. Summary in loss5_analysis.txt

=====
Running scenario: delay100
=====
Applying netem: delay 100ms
Starting tcpdump capture to pcaps/delay100.pcapng ...
Game running for 40 seconds...
Stopping clients and server...
Run complete. CSV logs should be created.
Stopping tcpdump ...
Running analysis with process_logs.py ...
Scenario delay100 finished. Summary in delay100_analysis.txt
```

Figure (4): Netem Output

6.3 Metrics Collected

- **Latency:** `recv_time - server_timestamp`
- **Jitter:** Variance in packet inter-arrival times
- **Position Error:** Euclidean distance between server (ground truth) and client perceived state

6.4 Automation

- `process_logs.py` interpolates timestamps and calculates spatial error.
- CSV outputs for analysis.

7.0 Example Use Case Walkthrough

Server Log

```
2025-12-11 03:08:32,830 - [STARTUP] Server initialized. Listening on port 1234...
2025-12-11 03:08:35,831 - [GAME] Spawned Event 2 (Type 1) at 2,17
2025-12-11 03:08:37,317 - [CONNECT] New connection accepted. Assigned Player ID 1 to ('127.0.0.1', 65318)
2025-12-11 03:08:37,444 - [CONNECT] New connection accepted. Assigned Player ID 2 to ('127.0.0.1', 65319)
2025-12-11 03:08:37,783 - [CLAIM] Player 1 successfully CLAIMED block at (0, 12)
2025-12-11 03:08:37,836 - [CONNECT] New connection accepted. Assigned Player ID 3 to ('127.0.0.1', 65320)
2025-12-11 03:08:37,912 - [CLAIM] Player 2 successfully CLAIMED block at (19, 1)
2025-12-11 03:08:38,061 - [CONNECT] New connection accepted. Assigned Player ID 4 to ('127.0.0.1', 65321)
2025-12-11 03:08:38,300 - [CLAIM] Player 3 successfully CLAIMED block at (7, 2)
2025-12-11 03:08:38,332 - [MOVE] Player 2 moved from (19, 1) to (18, 1)
2025-12-11 03:08:38,570 - [CLAIM] Player 4 successfully CLAIMED block at (6, 7)
2025-12-11 03:08:38,723 - [MOVE] Player 3 moved from (7, 2) to (6, 2)
2025-12-11 03:08:38,842 - [GAME] Spawned Event 4 (Type 1) at 3,13
2025-12-11 03:08:38,853 - [CLAIM] Player 1 successfully CLAIMED block at (0, 12)
```

Figure (5): Server Log Output

PCAP (Wireshark)

The screenshot displays the Wireshark network protocol analyzer interface. The top section shows a packet list with 15 entries, all of which are UDP packets from 127.0.0.1 to 127.0.0.1 on port 1234. The middle section shows the packet details for the selected packet (Frame 7), which is a UDP packet. The details pane shows the following information: Ethernet II (Type: IPv4), Internet Protocol Version 4 (Source: 127.0.0.1, Destination: 127.0.0.1), User Datagram Protocol (Source Port: 55203, Destination Port: 1234), and Data (28 bytes). The bottom section shows the raw packet data in hexadecimal and ASCII format.

Figure (6): Wireshark Screenshot